

Writing Files

CSCI 1101

2024-03-07

Reading Files

Last time we saw how to read the contents of a file using the `read` method and a `with` block that will handle closing the file for us automatically.

```
def file_read(path) :  
    """ (str) --> str """  
    with open(path) as file :  
        contents = file.read()  
        # no close necessary  
    return contents
```

Reading File Lines

We also saw how to read the lines of a file to a list of strings using the `readlines` method.

```
def file_lines(path) :  
    """ (str) --> list str """  
    with open(path) as file :  
        lines = file.readlines()  
        # no close necessary  
    return lines
```

Activity: File Statistics

Write a function `file_stats : (str) --> tuple [int, int, int]` that returns the number of *lines*, *words*, and *characters* contained in a file.

For example:

```
> file_stats("file_1.txt")  
(12, 83, 510)
```

Hint: you can compare the output of your function against that of the built-in program `wc` (for “word count”) on your computer.

File Access Modes

So far we have only been reading files, but we can write them as well.

To tell python what we want to do with a file we provide the `open` function with a string representing the desired **file access mode**.

```
open(<path> , <mode>)
```

Among these are the following:

read: "r" (the default assumed if you omit the mode)

write/truncate: "w" (overwrites existing file contents, *use with caution!*)

write/append: "a" (adds onto the end of existing file contents)

write/create: "x" (creates a new empty file)

The Read-Modify-Write Pattern

A common pattern of working with files is to:

- read data from some file,
- process the data somehow,
- write the data to some (either the same or a different) file.

```
def alphabetize (in_path , out_path) :  
    # alphabetizes the lines of a file  
    with open(in_path , "r") as in_file :  
        in_lines = in_file.readlines()  
    out_lines = sorted(in_lines)  
    with open(out_path , "x") as out_file :  
        for line in out_lines :  
            out_file.write(line)
```

Activity: Censoring Files

Write a function called `censor` that takes paths to an input and output file, a censored phrase and a replacement phrase, and copies the text from the input file to the output file but substituting every occurrence of the censored phrase by its replacement.

For example:

```
> censor('two_cities.txt', 'meh.txt' , 'was' , 'was not')
> print(file_read('meh.txt'))
It was not the best of times,
it was not the worst of times,
...
```

Character Encodings

A **character encoding** is a translation between single units of text (characters) and their computer representations (sequences of binary data).

CodeRunner admonishes you to always open files using the character encoding called “utf8”, which is a common **Unicode** encoding:

```
with open(infile_path, 'r', encoding='utf8') as infile :
```

```
with open(outfile_path, 'w', encoding='utf8') as outfile:
```

This encoding is already the default for *most* systems (including current versions of MacOS and Windows), and will soon become the required default for *all* systems (see PEP686).

In light of this, I do not require you to provide this redundant information, but if CodeRunner tells you to do so on an exercise, then you should.

To Do

Finish on *Project 2 - What Does the Fox Say?* on CodeRunner by midnight this Friday.

Finish *Homework 7: While loops and Files* on CodeRunner by 11:00AM Thursday, March 28.